

Ziemniak

- **Limit czasu:** 10 minut
- **Środowisko:** 1 GPU (≈16 GB VRAM), bez internetu
- **Rozmiar rozwiązania:** `solution.ipynb` ≤ 1 MB
- **Pamięć masowa:** 5 GB

Zadanie

Twój znajomy proponuje grę w zgadywanie. Jako sędzia wybiera jedno ukryte słowo z ustalonego słownika, a Ty musisz je znaleźć w co najwyżej 30 turach. W każdej turze sędzia porównuje dwa słowa i informuje, które z nich jest semantycznie bliższe ukrytemu słowu. Każda gra zaczyna się od ustalonej pary `lamp` vs `potato` (lampa czy ziemniak), ponieważ są to dwie z ulubionych rzeczy Twojego znajomego. Następnie Twój program proponuje jedno nowe słowo. Zwyczajca porównania zostaje zachowany i porównany z Twoją następną propozycją. Wygrywasz grę w chwili, gdy zaproponujesz dokładnie ukryte słowo. Przy porównywaniu wielkość liter nie ma znaczenia. Każde zaproponowane przez Ciebie słowo musi należeć do `dataset/vocabulary.json`.

Pełny przykład wraz z protokołem i wczytywaniem danych znajduje się w `solution.ipynb`. Możesz zmienić klasę `PublicEmbeddingPlayer`. Twój program jest inicjalizowany raz i rozgrywa każdą grę w jednym uruchomieniu; protokół tworzy nowy obiekt `PublicEmbeddingPlayer` na początku każdej gry.

Sędzia (judge)

Twój program wysyła do Sędziego jeden obiekt JSON, a Sędzia odpowiada jednym obiektem JSON.

Przykład działania, w którym ukryte słowo pokazano wyłącznie w celu objaśnienia protokołu:

```
Ukryte słowo: shovel          Początek zawsze ten sam: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Tury są indeksowane od 1 do 30.

Możliwe wartości `verdict` to `first`, co oznacza, że `word1` jest bliższe, `second`, co oznacza, że `word2` jest bliższe, lub `same`, co oznacza, że oba słowa są równie bliskie ukrytemu słowu.

`winner_word` to słowo, które zostaje zachowane do następnego porównania. W przypadku werdyktu `same` pierwsze słowo zostaje zachowane.

Zbiór danych

Wspólne dla każdego podziału (ang. split):

- `dataset/vocabulary.json` — 1602 unikalnych słów zapisanych małymi literami. Ukryte słowo jest zawsze jednym z nich.
- `dataset/public_embeddings.npy` — float32, kształt (1602, 2560). Wiersz `i` odpowiada słowu `i` w słowniku. Są to *publiczne* reprezentacje wektorowe (ang. embeddings); sędzia używa innych, prywatnych reprezentacji.

Podziały są zbiorami ukrytych słów:

Podział	Słowa	Odpowiedzi	Zastosowanie
<code>test_public</code>	120	✓ <code>dataset/test_public.json</code>	uruchamianie rozwiązania i samodzielna ocena wyniku
<code>test_leaderboard_a</code>	120	ukryte	ranking "na żywo"
<code>test_leaderboard_b</code>	120	ukryte	ranking końcowy

Nie ma podziału `train` — nic nie jest trenowane (ang. `fitted`) na podstawie oetykietowanych wierszy.

Dostarczone modele

Wraz z zadaniem dostarczane są dwa pretrenowane modele reprezentacji wektorowych (embeddingów), których można użyć:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Oba muszą zostać wczytane ze swojej lokalnej ścieżki; użycie Hugging Face hub id, takiego jak "BAAI/bge-m3", powoduje próbę pobrania modelu z internetu i kończy się niepowodzeniem, ponieważ ocenianie odbywa się offline. Każdy katalog zawiera działający (ang. `runnable`) plik `example.py` demonstrujący wykonanie offline.

Dostępne biblioteki: `numpy`, `torch`, `sentence-transformers`. Brak internetu, możliwości pobierania i innych pakietów.

Wyjście

Brak. Jest to zadanie interaktywne: Twoje rozwiązanie nie zapisuje pliku odpowiedzi; komunikuje się z sędzią przez `stdin/stdout` zgodnie z powyższym opisem.

Metryka

Gra rozwiązana w turze `t` otrzymuje wynik $1.0 - 0.02 \times \max(0, t - 10)$; gra nierozwiązana w ciągu 30 tur otrzymuje wynik 0. Zatem tury 1–10 dają wynik 1.00, tura 20 daje wynik 0.80, a tura 30 daje wynik 0.60.

Twój wynik za zadanie to średni wynik $\text{gry} \times 100$, pomiędzy 0.00 a 100.00.

Limit 10 minut jest obejmuje uruchomienie, przygotowanie oraz wszystkie 120 gier w zestawie testowym.

Jak przesłać rozwiązanie

1. Otwórz `solution.ipynb`, edytuj `PublicEmbeddingPlayer` i uruchom wszystkie komórki, aby upewnić się, że działa.
2. Opcjonalnie sprawdź je lokalnie: `python local_test.py solution.ipynb --limit 5`. Lokalny sędzia używa *publicznych* reprezentacji wektorowych, więc jego wynik jest jedynie wskazówką.
3. Zapisz `solution.ipynb`.
4. Otwórz kartę Git na lewym pasku bocznym JupyterLab.
5. *Dodaj* `solution.ipynb` *do obszaru przejściowego* (ang. *stage*) (ikona + obok niego).
6. Wprowadź opis commita i kliknij Commit.
7. Kliknij ikonę chmury ze strzałką w górę, aby wykonać push.
8. Wróć na tę stronę konkursu i kliknij Submit, używając opisu commita zgodnego z wcześniej podanym.

Prześlij dokładnie jeden plik o nazwie `solution.ipynb`, obejmujący wszelkie niezbędne przygotowania i inferencję (ang. *inference*).